

Confidence Estimation Module for LLM answer

Hussain Salih 2026
hussainsalih.github@gmail.com

Abstract

Traditional LLM suffer from uncertainty that can lead to hallucination. This study tries to give any LLM the ability to express its own confidence depending on measures whether internal or external like (token probability, self consistency, semantic similarity and self evaluation). Previous studies focus on calculating confidence scores based on metrics like AUROC or ECE and most of them focus on self evaluation, here we focus on providing model answers based on model confidence about the given answer then verbalising it.

Make a confidence estimation framework that converts any black-box LLM into a calibrated uncertainty-aware system.

Literature Review

1. Finetuning Language Models to Emit Linguistic Expressions of Uncertainty - 2024

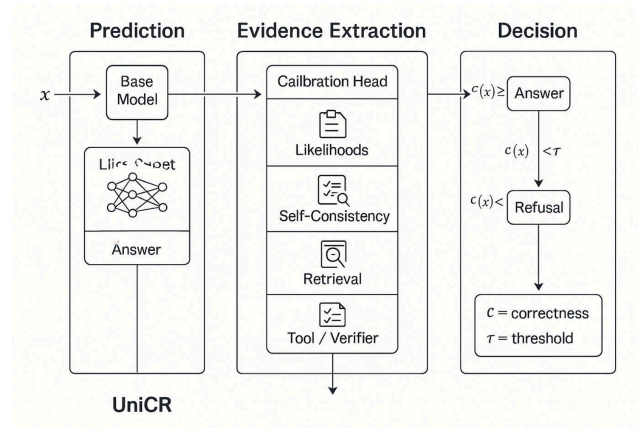
Supervised finetuning LLM on curated dataset to express uncertainty verbally rather than a number, performing True/False self-evaluation task through steps

1. Few-Shot prompting 8-16 examples and 1-2k samples to train isotonic regressor (frequently used to adjust predicted probabilities) to ensure numerical accuracy.
2. Self-Evaluation : Generate answer and ask model if it's answer is True or False , Extract Probability (model internal probe of the token true) dividing probability of True by the sum of True and False
3. Post processing to get near-perfect calibration isotonic regression to ensure a model claiming 80% is actually 80% correct of the time ,using model self reported (beliefs) and truth values.
4. Mapping Numbers to linguistic expressions, they found that using post-fixing expression is more effective than prefix or interleaved.

Research link : <https://arxiv.org/pdf/2409.12180>

2. Trusted Uncertainty in Large Language Models: A Unified Framework for Confidence Calibration and Risk-Controlled Refusal - 2025

UniCR added an Evidence Extraction layer to the transformer architecture; if a model is "black-box" (API-only) and does not provide log-probabilities, the layer can ignore likelihood features and rely entirely on self-consistency and retrieval signals.



Research link : <https://arxiv.org/pdf/2509.01455>

LLMs often generate fluent answers even when they are wrong. In domains such as healthcare, finance, education, robotics, and scientific research, Also it may have serious consequences, such as the user's excessive trust in artificial intelligence leading to the avoidance of the real source of knowledge (books) and taking all information from it regardless of its accuracy because the artificial intelligence has been trained on diverse data and different opinions on the same issues (politics, history, etc.).

Proposed Method

Using 4 signals based on model answer

1. Token Probability : The raw softmax probability of the generated sequence, serving as a baseline measure of predictive certainty.

2. Self-Consistency (self: The agreement across multiple sampled completions (e.g., 5-10 paths). High variance indicates stochastic instability.

3. Semantic Similarity : The coherence between the generated answer and verified truths (or consensus among samples), ensuring the meaning remains stable across variations.

4. Self Evaluation : $(P(\text{True})/P(\text{False}))$ from logits)

Find confidence score from these signals (normalized average), the equation :

$$C = \frac{\sum_{i=1}^n \frac{p_i}{\epsilon_i}}{\sum_{i=1}^n \frac{1}{\epsilon_i}}$$

Where :

p_i = confidence signal

ϵ_i = uncertainty / error (ECE)

Then using verbal descriptions.

Verbal Description : It's not just confidence score , but it's confidence-degree answer , For example :

- 1.0 -> 'certain'
 - 0.87-0.9 -> 'almost certain', 'high likely'
 - 0.75-0.86 -> 'very good chance', 'likely'
 - 0.63 - 0.74 -> 'I believe', 'probably', 'probable'
 - 0.40 - 0.62 'chances are about even'
 - 0.18 - 0.39 'I doubt', 'unlikely'
 - 0.02 - 0.17 'little chance', 'chances are slight', 'improbable', 'highly unlikely'
 - 0.00 - 0.01 'impossible'
-

Applications

1. Hallucination control : the most immediate use. When token probability and self-consistency are both low, the model is essentially guessing.

5. Interpretability : what the model "actually thinks" versus what it says.

Implementation

Step 1 :

Get the 4 signals from LLM (**Qwen2.5-1B-Instruct**) based on **TruthFulQA** dataset with 500 samples , then export the result as json (question, answer, prediction , p_internal , p_consistency , p_semantic, p_self_eval).

Step 2 :

Take those results (json) and use LLM to act as a judge to find if the model gets the correct answer or not as a boolean value (0,1) then make a json file for the 4 signals and correctness of the model.

Step 3 :

Use fusion equation for these 4 signals , then adding verbal description as label from confidence score, Inferences add verbal description on postfix of answer depends on first study mentioned in literature review by google deep mind , finds the postfix is better than prefix or interleaved for future fine-tune to allow models to focus on the answer more than the statement.

Step 4 :

Fine-tune the model with these data to answer with confidence state, using QLora with 2K dataset and to validate it using out-of-fold cross validation technique to ensure not biased.

Results

__Experiment #0__

It fails using only **500** samples from **truthfulQA** , table order based on **AUROC**:

Method	AUROC ↑	ECE ↓	Brier ↓
TokenProb	0.5691	0.4231	0.2401

Results show most effective signals are token probability, and the others act like noise depends on this table :

And how drop in **AUROC** when given signal removed :

Removed Feature	AUROC Drop
-TokenProb	0.111702

Experiment #1

Use 5 samples for self-consistency and 2000 samples from **TriviaQA** dataset,
Model : **Qwen2.5-1.5B-Instruct**

Method	AUROC	ECE	Brier
FullFusion	0.676508	0.409202	0.234116
-SemanticSimilarity	0.676277	0.409508	0.234003
-SelfConsistency	0.676277	0.409508	0.234003
-TokenProb	0.672702	0.410005	0.235129
SelfConsistency	0.640180	0.417429	0.238716
SemanticSimilarity	0.640180	0.417429	0.238716
-SelfEval	0.639373	0.417071	0.238836
SelfEval	0.613366	0.434999	0.246891
TokenProb	0.550686	0.439213	0.244585

And what the remove of each signals effects :

Removed Feature	AUROC Drop
-SelfEval	0.037135
-TokenProb	0.003806
-SelfConsistency	0.000231
-SemanticSimilarity	0.000231

Then use normalized average equation I discussed above to get confidence score , verbalize it in new dataset ,

Fine-tune the model with QlorA techniques , and result shows :

Step	Training Loss
10	1.419517
20	0.226940
30	0.149778
40	0.137347
50	0.126462

60	0.121923
70	0.119010
80	0.126690

With Evaluation loss: 0.11381830275058746

Save the fine-tuned model in huggingFace Model , then create space and use this model with code to show result to end users,

Example of result /

Source Code : <https://github.com/Shubbair/LLM-Confidence-Calibration-Module/>

Model Source : <https://huggingface.co/Shubbair/Qwen2.5-1.5B-Confidence-Calibration>

Live Experiment : <https://huggingface.co/spaces/Shubbair/Confidece-Clibaration-Qwen2.5-1.5B>

Future work

Real-time confidence estimator for LLM outputs (API-ready) , train ML for these metrics then predict for new case LLM samples.